Software Hackathon 2026

Domain:**Software Development** (Math application)

Team Name	LocalHost one
Team Leader	J V Karthikeyan
Team Members	Dheepak Adhithya S

Software Requirements Specification (SRS)

Math Learning Management System (Helix)

Version: 1.0
Document Type: Software Requirements Specification (SRS)
Prepared By: Karthi Keyan,Dheepak Adhithya
Project: Math Learning Management System (SrS)

Table of Contents

---

1. Introduction

1.1 Purpose

The Math Learning Management System (Math LMS) is a web-based educational platform designed to facilitate effective mathematics learning through interactive learning modules, automated assessments, AI-powered tutoring, and comprehensive performance analytics. The system connects administrators, teachers, and students within a secure digital environment to streamline educational processes and improve learning outcomes.

1.2 Scope

Math LMS enables educational institutions and independent educators to conduct online mathematics education efficiently. The platform provides:

- Secure user authentication
- Role-based authorization
- Digital classroom management
- Interactive learning modules
- Quiz and assessment management
- Automated grading

- Student progress tracking
- AI-powered mathematical assistance
- Performance analytics
- Administrative management

1.3 Objectives

The primary objectives of the system are:

- Improve mathematics education through digital learning.
- Reduce manual effort in assessment and grading.
- Provide instant performance feedback.
- Enable personalized AI-assisted learning.
- Ensure secure access using role-based authentication.
- Provide scalable and maintainable architecture.

---

2. Overall Description

2.1 Product Perspective

Math LMS follows a three-tier client-server architecture consisting of:

- Presentation Layer (React Frontend)
- Business Logic Layer (Spring Boot Backend)
- Data Layer (MySQL Database)

All components are containerized using Docker and orchestrated through Docker Compose.

2.2 User Classes

Administrator

Responsibilities include:

- User management
- Teacher approval

- Student management
- Dashboard monitoring
- Platform analytics
- System configuration

Teacher

Responsibilities include:

- Managing learning modules
- Creating quizzes
- Managing questions
- Evaluating student performance
- Viewing analytics
- Monitoring student progress

Student

Responsibilities include:

- Viewing learning materials
- Attempting quizzes
- Viewing quiz results
- Monitoring learning progress
- Using AI tutor
- Updating personal profile

2.3 Operating Environment

Frontend

- React
- Vite
- Tailwind CSS
- Modern Web Browsers

Backend

- Java 21
- Spring Boot
- Maven

Database

- MySQL 8

Deployment

- Docker
- Docker Compose
- Nginx

---

3. System Features

3.1 Authentication Module

Features:

- User Registration
- Secure Login
- JWT Authentication
- Password Encryption
- Role-Based Authorization
- Logout

3.2 User Management

Administrator can:

- Add users
- Update users
- Delete users
- Assign roles
- Manage permissions

Users can:

- Update profile
- Change password
- View profile information

3.3 Learning Module

Teachers can:

- Create learning modules
- Edit modules
- Delete modules
- Organize chapters
- Upload resources

Students can:

- Browse modules
- Read lessons
- Access learning resources

3.4 Quiz Management

Teachers can:

- Create quizzes
- Add questions
- Configure quiz duration
- Publish quizzes
- Review submissions

Students can:

- Attempt quizzes
- Submit answers
- Receive immediate evaluation
- View quiz history

3.5 AI Tutor

Students can:

- Ask mathematics questions
- Receive explanations
- Solve mathematical problems
- Clarify concepts
- Obtain guided learning support

3.6 Dashboard

Admin Dashboard

Displays:

- Total users
- Teacher statistics
- Student statistics
- Platform analytics
- Activity reports

Teacher Dashboard

Displays:

- Student performance
- Quiz statistics
- Learning progress
- Assessment reports

Student Dashboard

Displays:

- Learning progress
- Completed modules
- Quiz scores
- Performance history

---

4. External Interface Requirements

4.1 User Interface

The user interface shall provide:

- Responsive design
- Modern dashboard
- Mobile-friendly layout
- Simple navigation
- Interactive components
- Accessible forms
- Real-time notifications

4.2 Hardware Interface

Minimum Requirements:

- Internet-enabled computer
- Smartphone or tablet
- Modern browser
- Stable internet connection

4.3 Software Interface

Frontend communicates with backend through REST APIs using Axios.

Backend communicates with:

- MySQL Database
- JWT Authentication Services
- AI Integration Service

4.4 Communication Interface

Communication uses:

- HTTP/HTTPS
- REST APIs
- JSON Data Format

---

5. Functional Requirements

FR-1 User Authentication

The system shall:

- Register users
- Authenticate users
- Generate JWT tokens
- Validate JWT tokens
- Encrypt passwords
- Authorize user roles

FR-2 User Management

The system shall:

- Create user accounts
- Update user information
- Delete user accounts
- Retrieve user details
- Assign roles

FR-3 Learning Module Management

Teachers shall:

- Create learning content
- Edit learning content
- Delete learning content
- Publish modules

Students shall:

- View modules
- Access content

FR-4 Quiz Management

Teachers shall:

- Create quizzes
- Configure quizzes
- Publish quizzes
- Review submissions

Students shall:

- Attempt quizzes
- Submit responses
- View scores

FR-5 Assessment Evaluation

The system shall:

- Automatically evaluate quizzes
- Calculate scores
- Store results
- Generate reports

FR-6 AI Assistance

The system shall:

- Accept mathematical queries
- Process AI requests
- Return intelligent responses
- Assist in concept understanding

FR-7 Reporting

The system shall generate:

- Student reports
- Quiz reports
- Teacher reports
- Performance analytics

---

6. Non-Functional Requirements

Performance

The system shall:

- Respond within acceptable time.
- Support concurrent users.
- Optimize database operations.
- Minimize API latency.

Reliability

The system shall:

- Maintain data consistency.
- Recover from failures.
- Ensure continuous availability.

Security

The system shall:

- Encrypt passwords using BCrypt.

- Secure APIs using JWT.
- Restrict access using user roles.
- Validate all inputs.
- Prevent unauthorized access.

Scalability

The system shall:

- Support future feature additions.
- Handle increasing users.
- Support cloud deployment.
- Maintain modular architecture.

Maintainability

The system shall:

- Follow layered architecture.
- Use reusable components.
- Maintain clean code.
- Follow RESTful API standards.

Usability

The system shall provide:

- Simple navigation
- Responsive UI
- User-friendly dashboards
- Minimal learning curve

---

7. System Architecture

```
        Users
          |
   React + Vite Frontend
          |
     Axios REST Calls
          |
   Spring Boot Backend
          |
Controller → Service → Repository
          |
  Spring Data JPA (Hibernate)
          |
      MySQL Database
          |
 Docker Compose Deployment
```

---

8. Database Design

The database consists of relational entities including:

- User
- Role
- StudentProfile
- Teacher
- LearningModule
- Assessment
- Quiz
- Question
- QuizResult

Relationships:

- One Role → Many Users
- One Teacher → Many Learning Modules
- One Learning Module → Many Quizzes
- One Quiz → Many Questions
- One Student → Many Quiz Results

---

9. Security Requirements

The application implements:

- JWT Authentication
- Spring Security
- BCrypt Password Hashing
- Stateless Authentication
- Role-Based Access Control (RBAC)
- Input Validation
- Secure REST Endpoints
- Exception Handling
- Authentication Filters
- Authorization Middleware

---

10. Technology Stack

Layer	Technology
Frontend	React 18
Build Tool	Vite
Styling	Tailwind CSS
State Management	Zustand
Routing	React Router DOM
HTTP Client	Axios
Backend Language	Java 21
Backend Framework	Spring Boot 3.2.4
ORM	Spring Data JPA (Hibernate)
Security	Spring Security + JWT
Database	MySQL 8
Validation	Spring Validation
DTO Mapping	MapStruct

Boilerplate Reduction	Lombok
API Documentation	SpringDoc OpenAPI
Build Tool	Maven
Web Server	Nginx
Containerization	Docker
Orchestration	Docker Compose
Version Control	Git
Repository	GitHub

---

11. Deployment Architecture

The system is deployed using Docker Compose.

Deployment Components:

- React Frontend Container
- Spring Boot Backend Container
- MySQL Database Container
- Docker Network
- Nginx Reverse Proxy

Deployment Flow:

```
Client Browser
    │
    ▼
Nginx Container
    │
    ▼
React Frontend
    │
Axios REST API
    │
    ▼
Spring Boot Backend
    │
    ▼
```

MySQL Database

---

12. Future Enhancements

The following features are planned for future releases:

- Video lecture support
- Assignment submission portal
- Live virtual classrooms
- Discussion forums
- Certificate generation
- Parent dashboard
- Mobile application
- Push notifications
- Personalized AI learning recommendations
- AI-based performance prediction
- Gamification with badges and achievements
- Leaderboards
- Attendance management
- File sharing and collaborative learning
- Multi-language support

---

13. Conclusion

The **Math Learning Management System (Math LMS)** provides a modern, secure, and scalable platform for mathematics education. By integrating role-based access control, automated assessments, AI-assisted tutoring, and comprehensive learning analytics, the system enhances both teaching and learning experiences. Its modular architecture, RESTful APIs, containerized deployment, and responsive user interface make it suitable for educational institutions, online learning environments, and future expansion with advanced features such as live classes, personalized learning, and predictive analytics.